ICT 1062 Object-Oriented Programming

Individual Final Assignment: Java-Based GUI Sales System

Name: D.M.K.H. Dasanayaka
Index number: AS20240946
GitHub repository link: https://github.com/kDash22/projectSem2

1.INTRODUCTION

This project is a Sales Management System developed using Java and MySQL. It is designed to manage user, customer, products and sales transactions for a small-scale retail business. The system provides a graphical user interface for data entry, retrieval and management.

2.SYSTEM OBJECTIVES

- To allow users to log in securely with role-based access control
- To manage product information including add, update, delete, and view operations
- To manage customer records with proper validation of input data
- To create and process sales transactions linked with customers and items
- To generate invoices and calculate total amounts for each sale
- To store and retrieve sales data from a relational database using DAO classes
- To maintain data integrity through controlled database operations and transactions
- To ensure separation between user interface, business logic, and database access layers

3.REQUIREMENT ANALYSIS

FUNCTIONAL REQUIREMENTS

Users

• Ability to log in with role-based access (e.g. Admin, Clerk)
• Ability to add, delete, and manage users through the system interface
• Ability to perform authorized operations based on assigned roles
• Ability to view all users stored in the database

Customers

• Ability to add, delete, and manage customer records through the system interface
• Ability to search for a customer using customer ID
• Ability to view all customers stored in the database

Products

• Ability to add, delete, and manage product records through the system interface
• Ability to search for products using product ID
• Ability to view all products stored in the database
• Ability to handle unit types for selling goods (e.g. PIECE, WEIGHT)

Sales

• Ability to create, delete, and manage sales through the system interface
• Ability to link customers, sales, and sale items internally within the system
• Ability to search for sales using sale ID
• Ability to view all sales stored in the database
• Ability to display invoices when a sale is confirmed or retrieved by ID

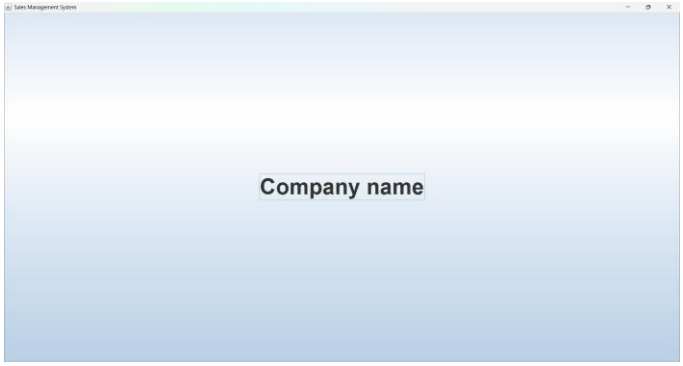
NON-FUNCTIONAL REQUIREMENTS

• The system ensures secure access through authentication and role-based authorization.
• The system maintains data integrity during all database transactions.
• The system offers a user-friendly, simple graphical interface.
• The system handles errors without crashing or losing data.
• The system ensures consistent communication between GUI, DAO, and database layers.

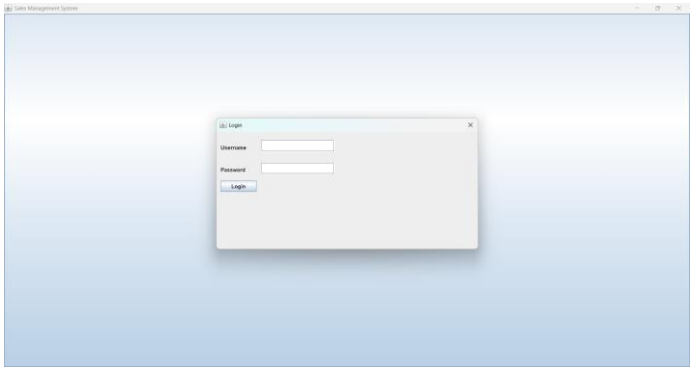
4.GUI SCREENSHOTS

The application starts with this window. The company name can be changed by changing a global variable inside the system, which can be located in the Global class.

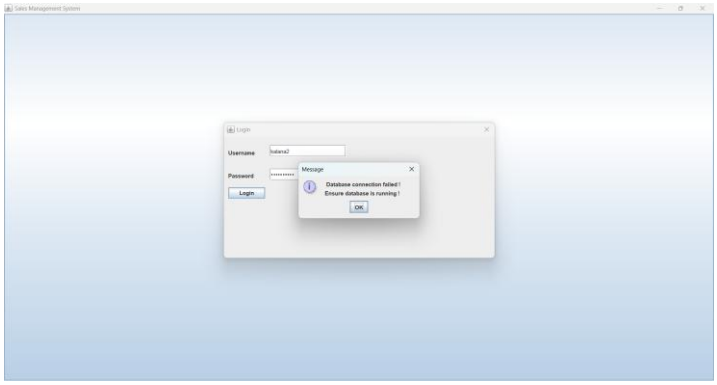
Users can access the log in screen by clicking anywhere on the window which acts like a huge button that activates the login screen.



The login window will appear after clicking somewhere on the window.

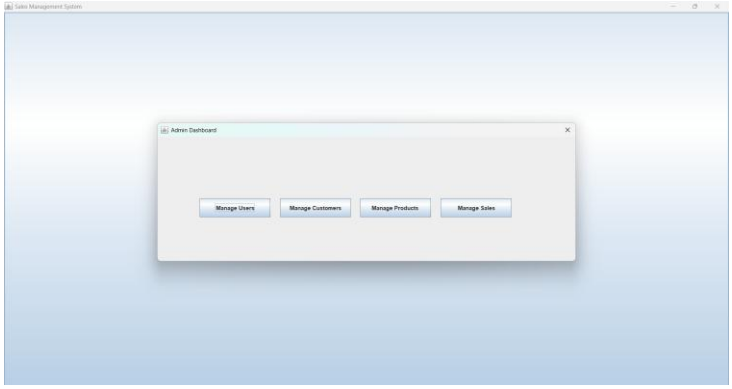


If the database is not active and running the login attempt will result is a error pop up.



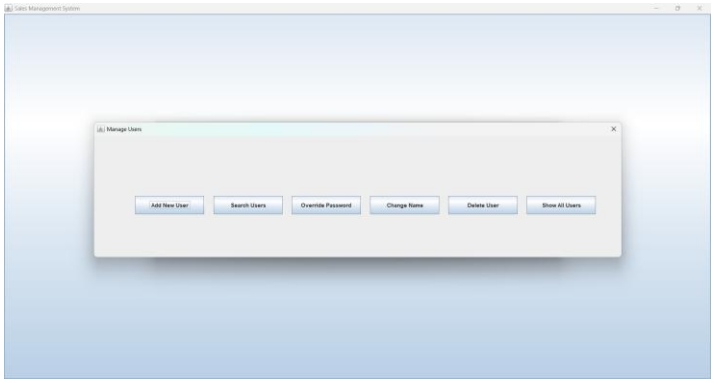
After a successful login a success message will pop up and depending on the user's role the relevant dashboard will be opened.

Admin Dashboard.

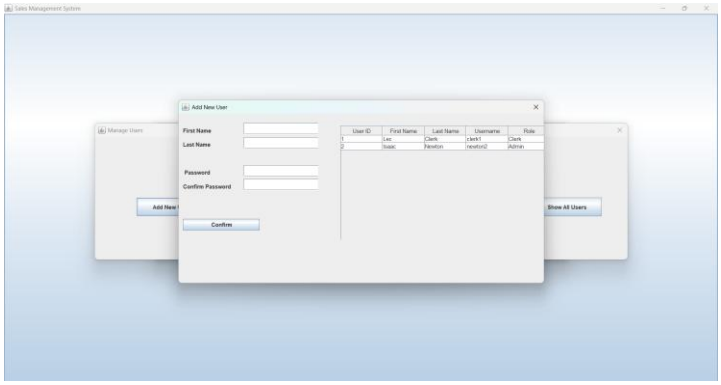


By clicking on any of the buttons shown the Admin can manage various areas relevant to the system.

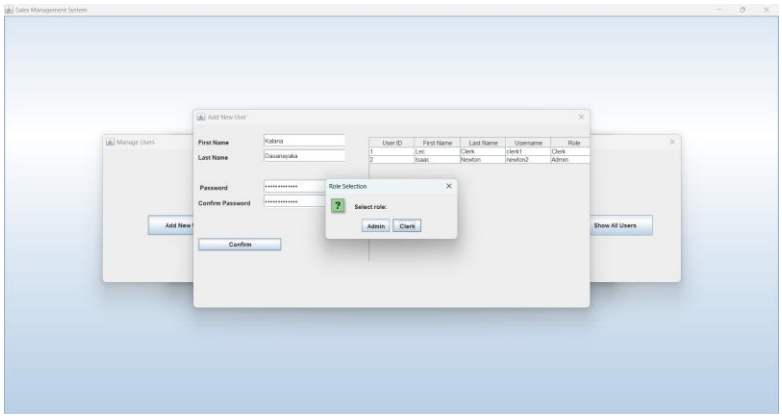
User management.



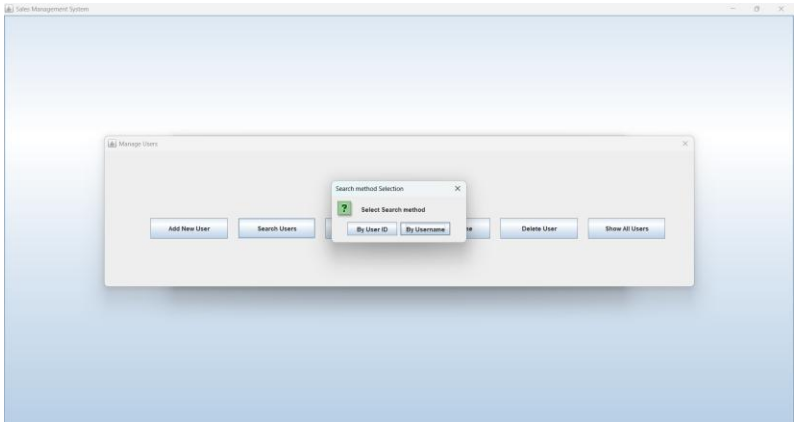
Add new user interface.



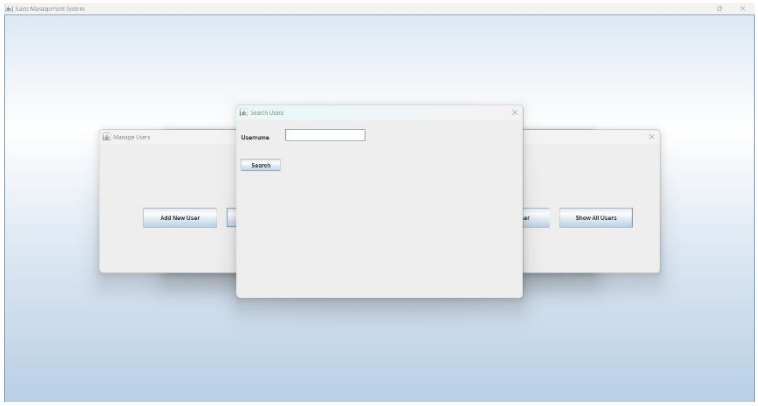
After correctly filling out the form and clicking confirm, a pop up show up with the role choices and the Admin can choose accordingly



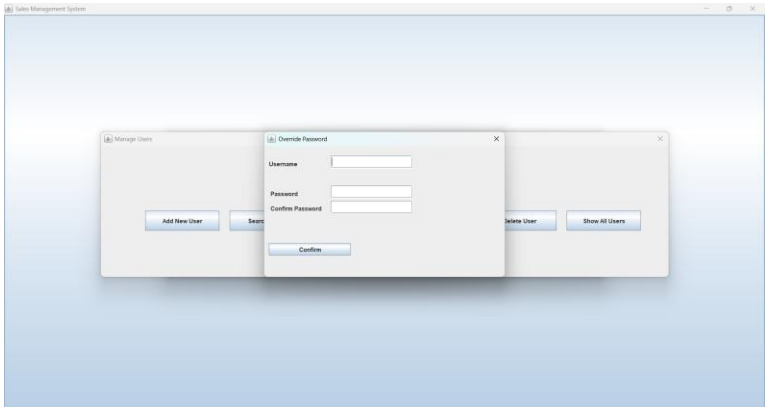
The search user button, after being clicked offers a choice of searching by username or user id by a pop up.



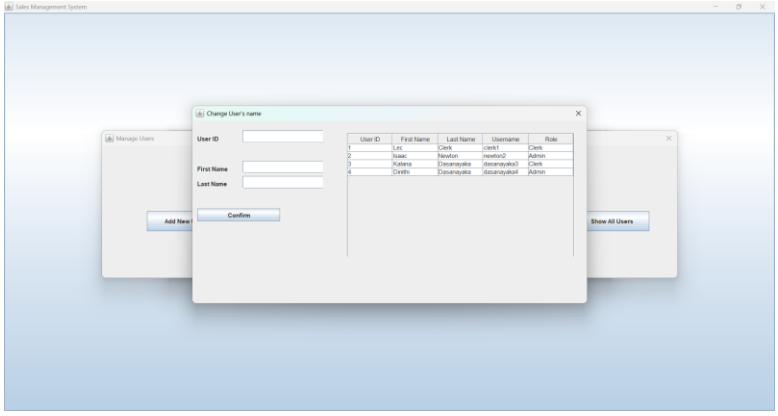
Search user interface is the same for both choice with the only change being the label Infront of the input field.



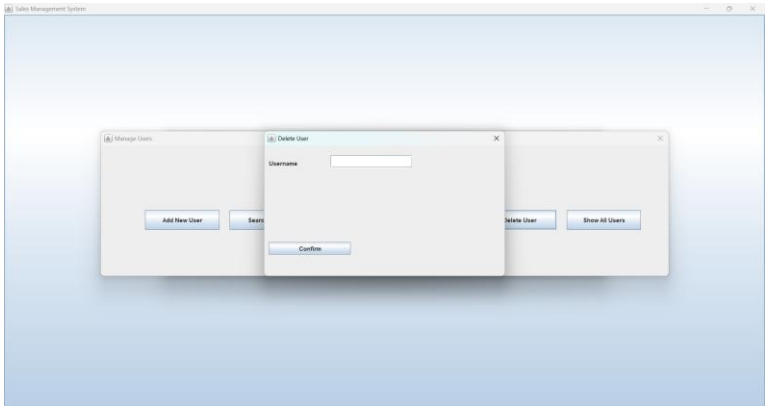
Override password button lets the Admin change any users password, helpful if someone forgets the password or a password gets compromised.



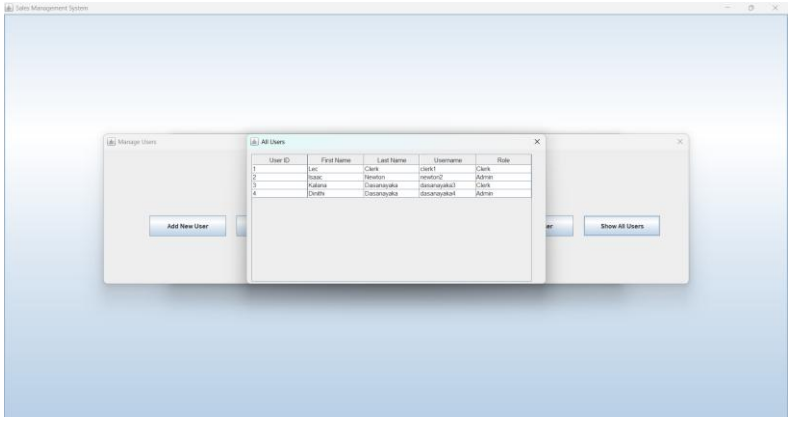
Change name button lets an Admin change another user's name.



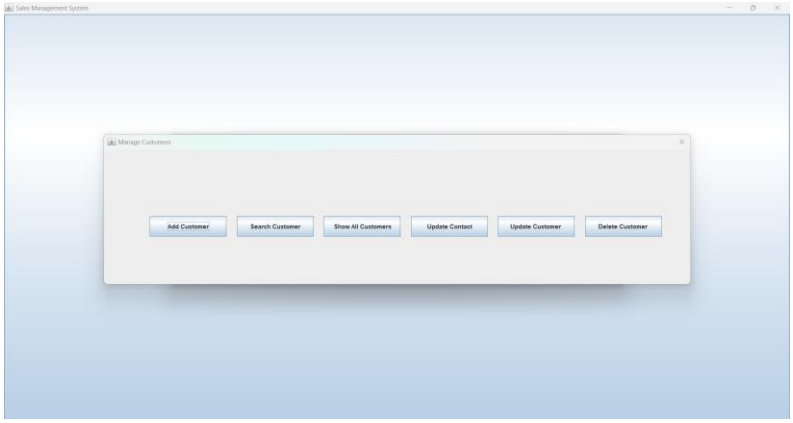
Delete user interface.



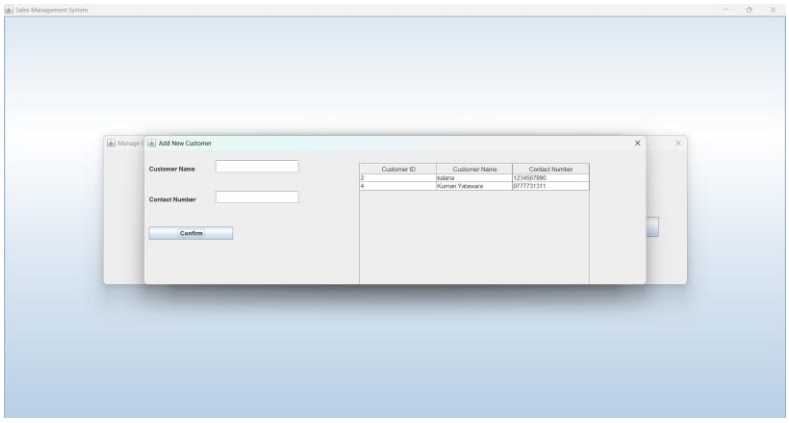
Show all users interface.



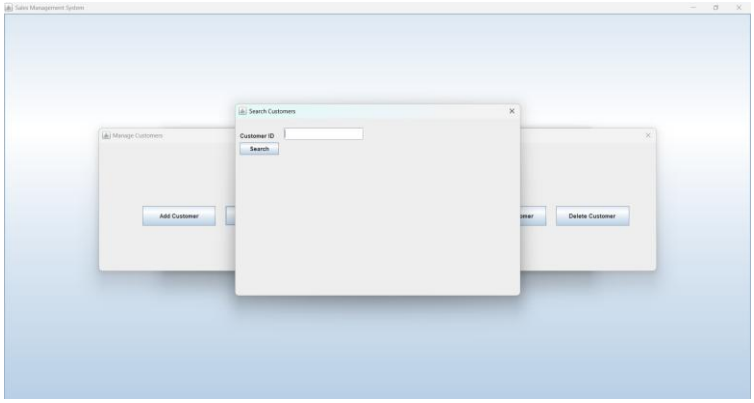
Customer management interface.



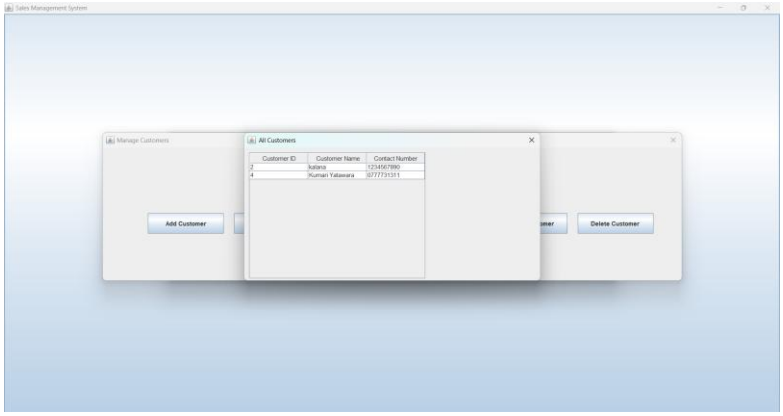
Add customer interface.



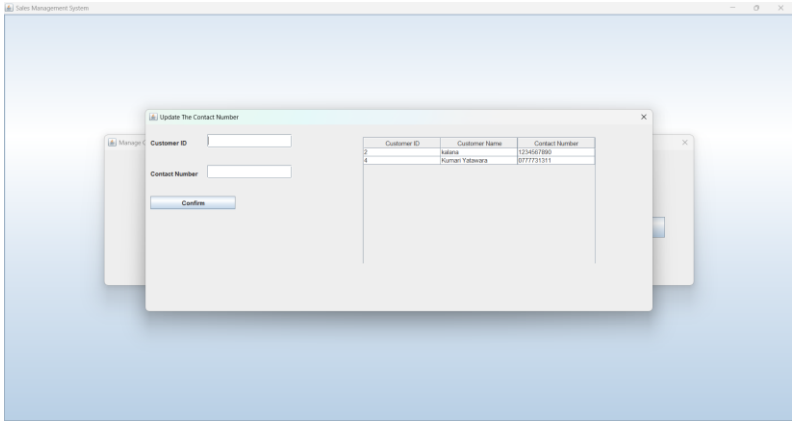
Search customer interface.



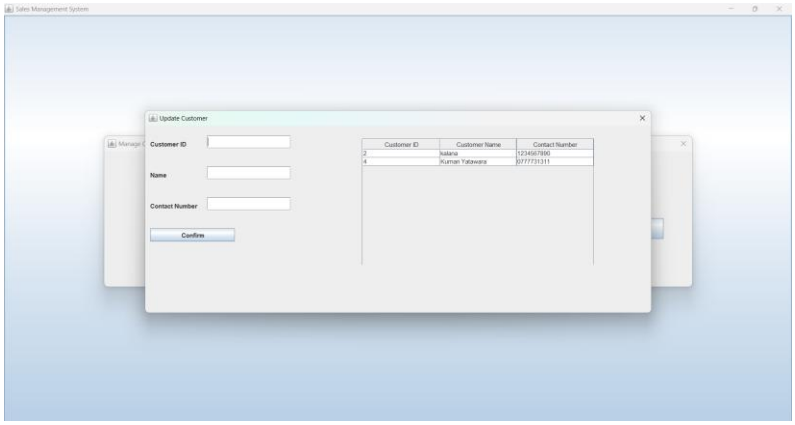
Show all customers interface.



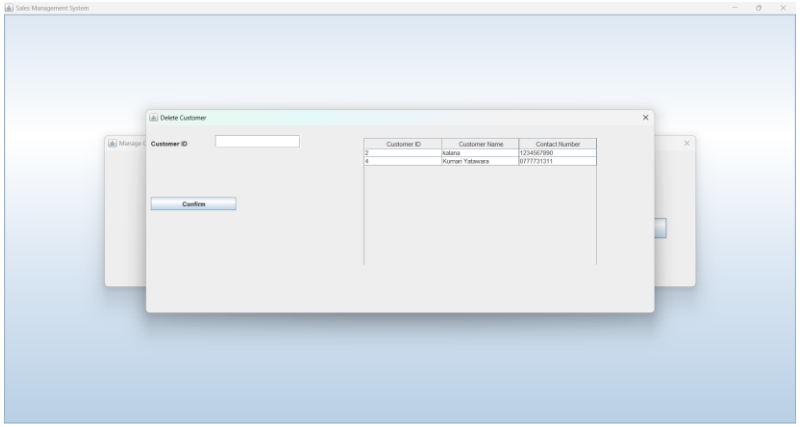
Update contact interface.



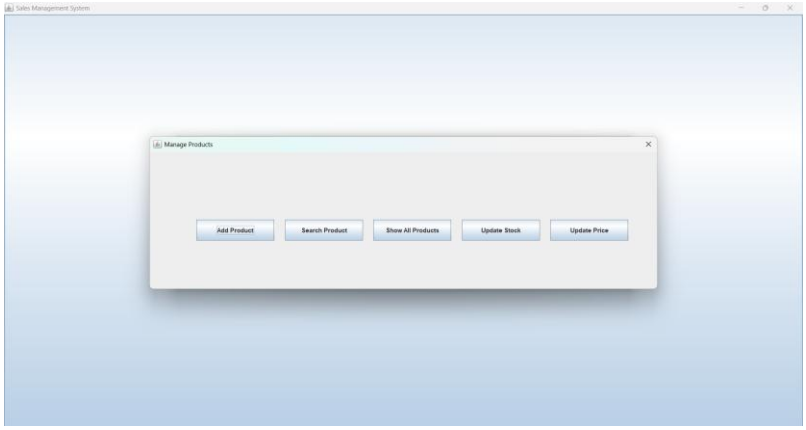
Update customer interface can be used to change all the information about a customer, useful if a customer wants to transfer the ownership of the account to someone else.



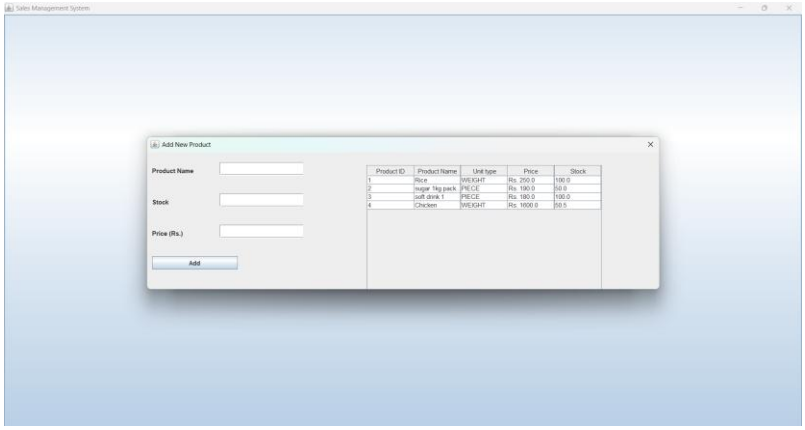
Delete customer interface.



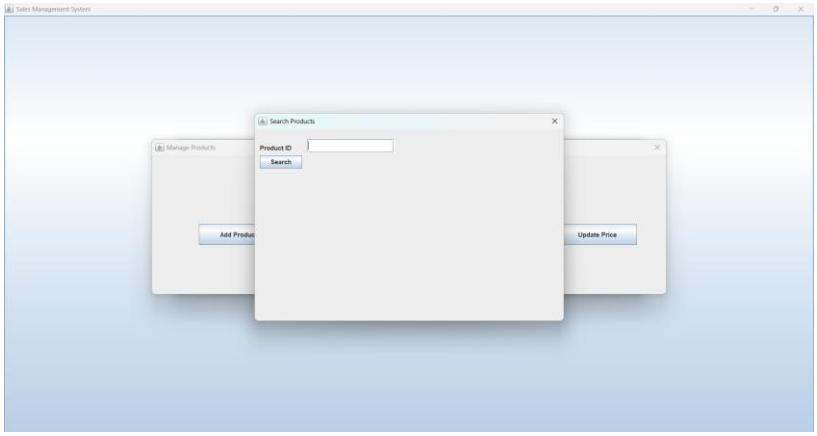
Manage product interface.



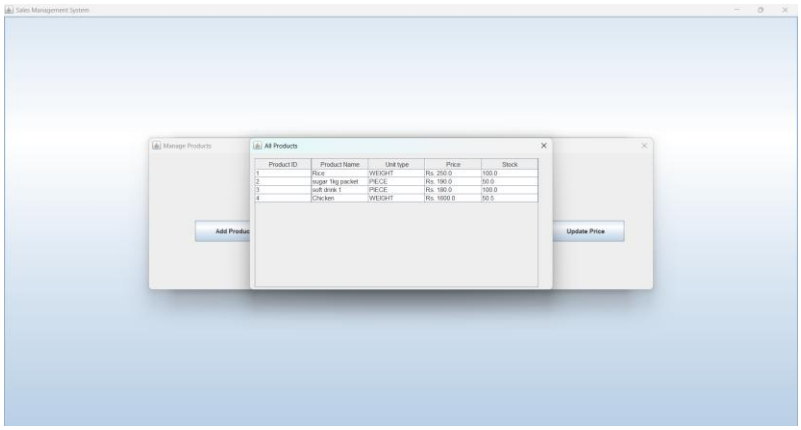
Add product interface.



Search products interface.

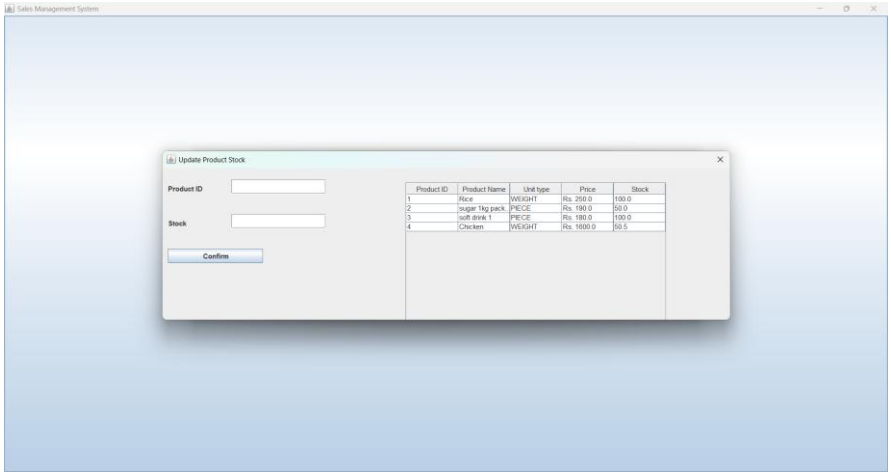


Show all products interface.

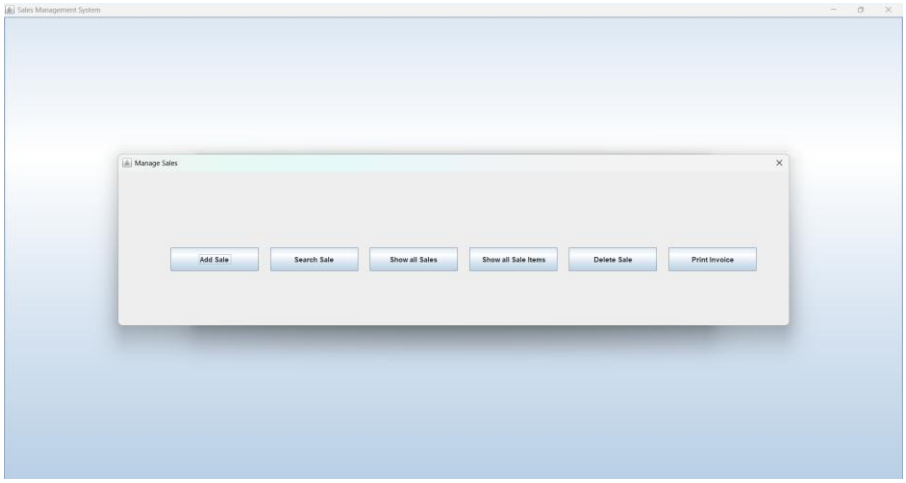


Update price and update stock interfaces differ only in name and the label before the input field which is price and stock for the windows respectively.

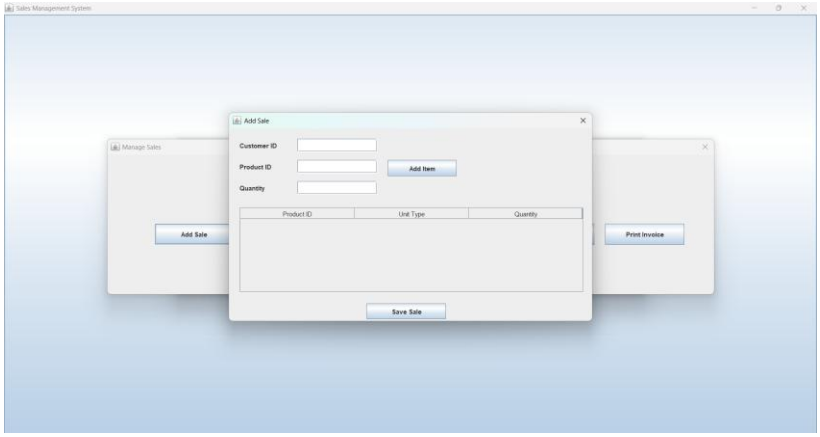
Update stock interface.

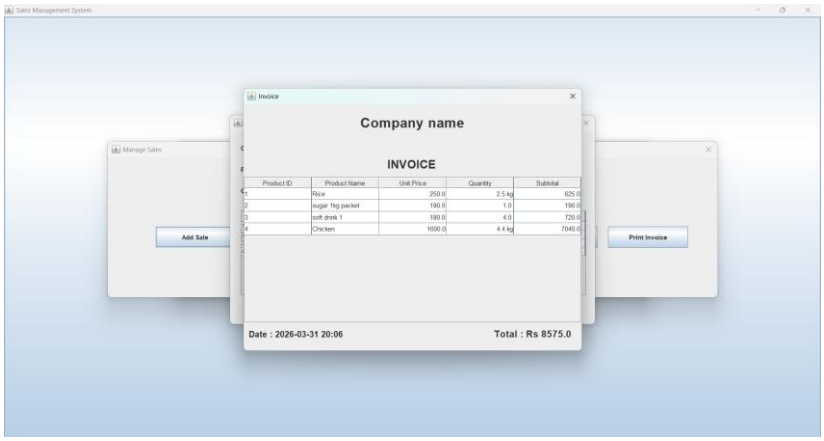


Manage sales interface.

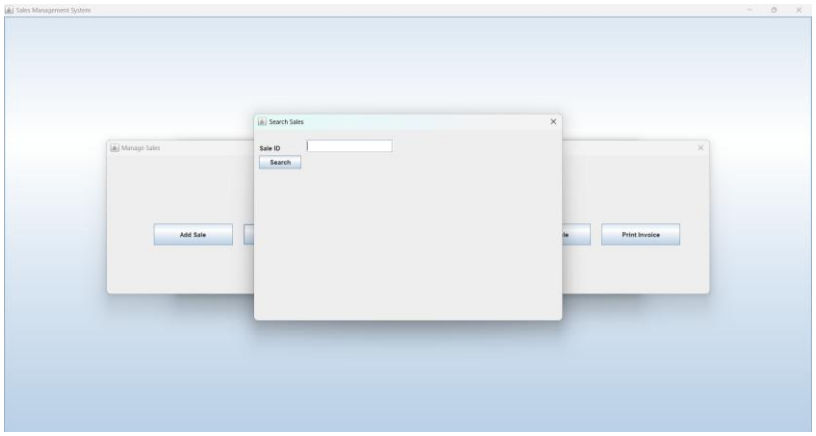


Add sale interface lets you add sales items to the sale and at the moment the save sale is clicked, a pop up will ask if the admin wants to print the invoice or not, which will show the invoice on the screen. If the user clicks yes, the invoice will be shown.

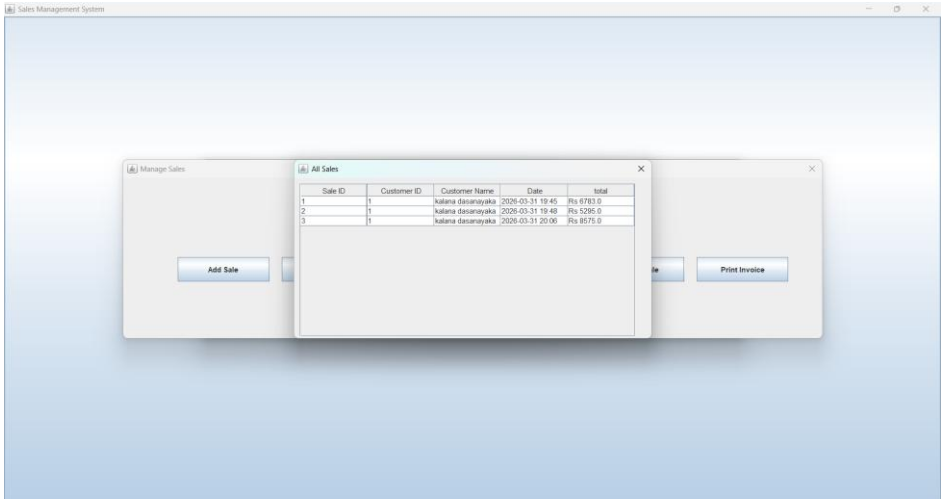




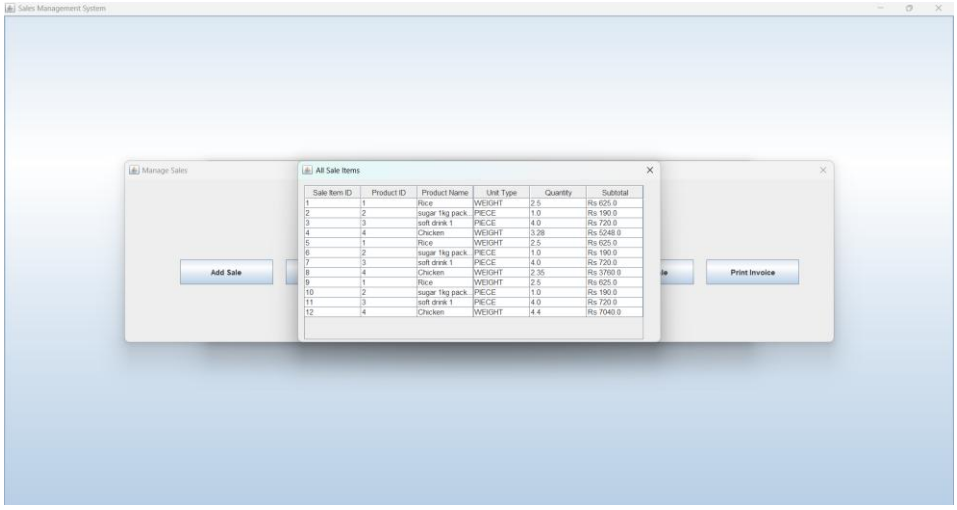
Search sale interface.



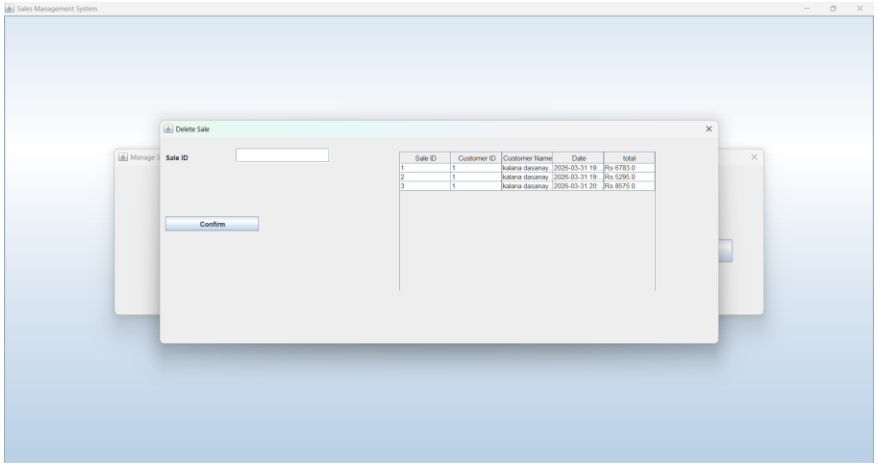
Show all sales interface.



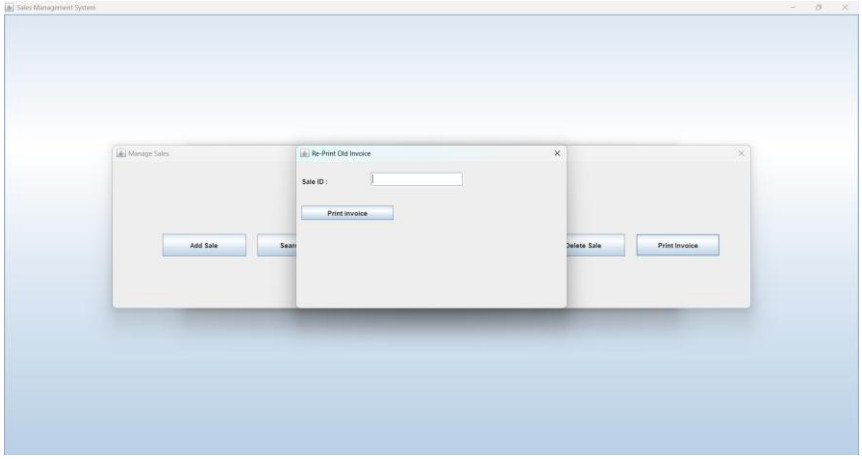
Show all sale items interface, this will show all the items in the sale_items tables of the database.



Delete sale interface.

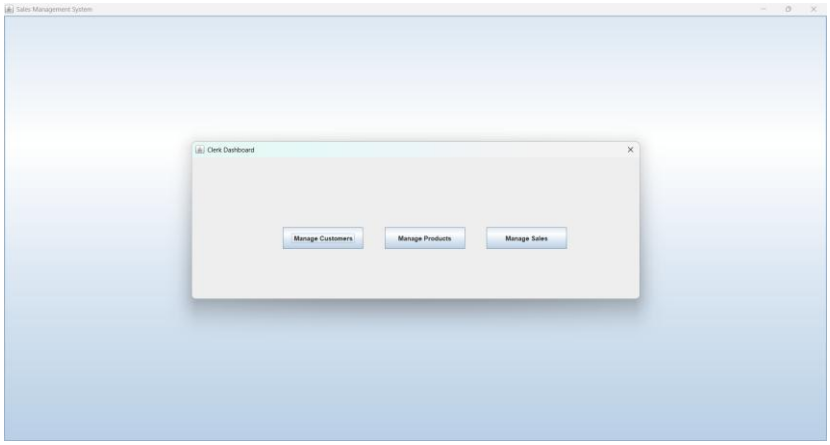


Print invoice interface can be used to print invoices of old sales.

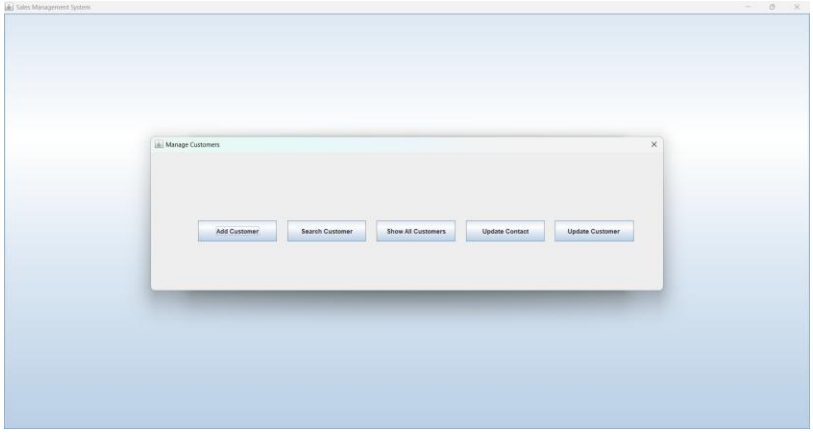


Clerk Dashboard

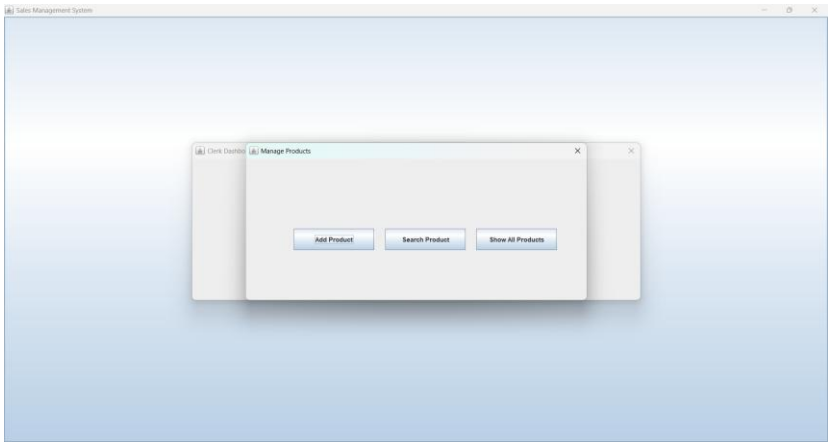
Clerk dashboard differs from the admin dashboard in minor ways as some privileges has been withdrawn from the Clerk role.



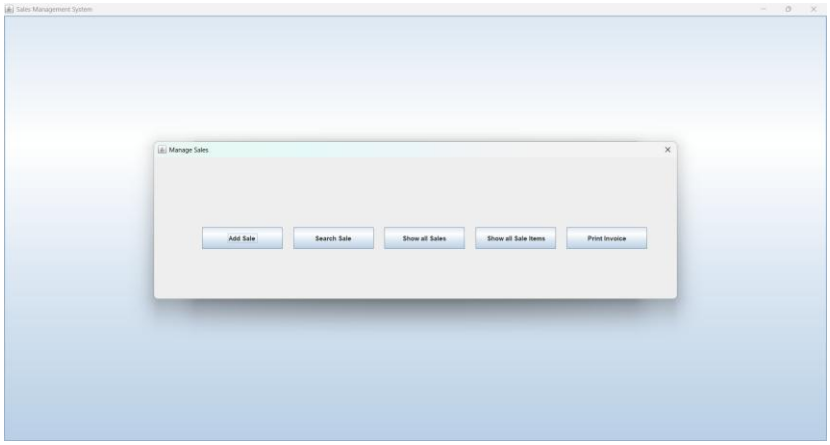
Manage customers interface.



Manage products interface.

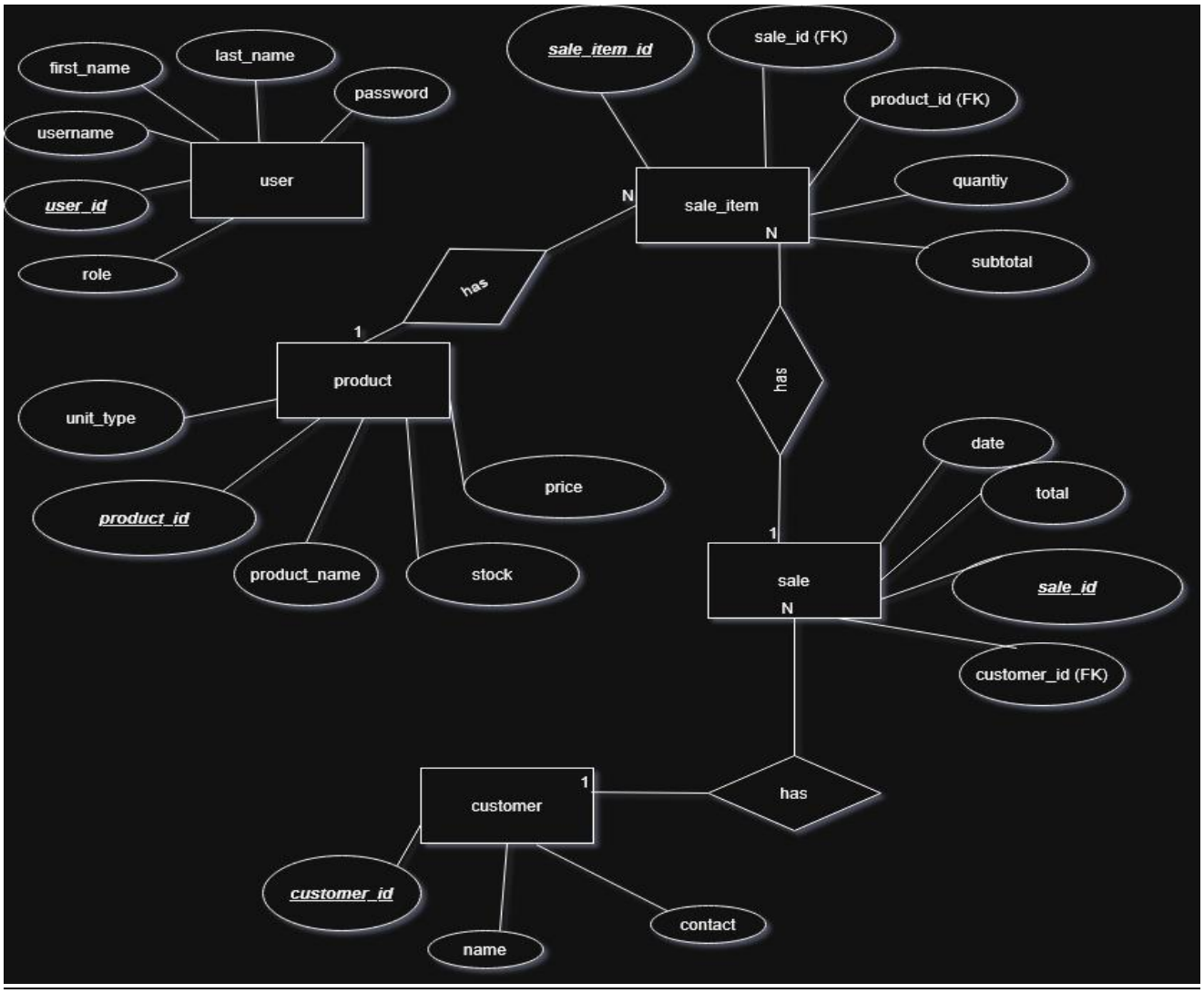


Manage sales interface.



5.DATABASE DESIGN

ER Diagram



Tables

customers

Column	Type	Constraints
customer_id	INT	PK, AUTO_INCREMENT
name	VARCHAR(255)	NOT NULL
contact	VARCHAR(255)	NOT NULL

users

Column	Type	Constraints
user_id	INT	PK, AUTO_INCREMENT
first_name	VARCHAR(255)	NOT NULL
last_name	VARCHAR(255)	NOT NULL
password	VARCHAR(255)	NOT NULL
role	ENUM	NOT NULL (Admin, Clerk)
username	VARCHAR(255)	UNIQUE, NOT NULL

products

Column	Type	Constraints
product_id	INT	PK, AUTO_INCREMENT
product_name	VARCHAR(255)	NOT NULL
price	DOUBLE	NOT NULL
stock	DOUBLE	NOT NULL
unit_type	ENUM	NOT NULL (WEIGHT, PIECE)

sales

Column	Type	Constraints
sale_id	INT	PK, AUTO_INCREMENT
customer_id	INT	FK → customers(customer_id), NULL
date	DATETIME	DEFAULT CURRENT_TIMESTAMP
total	DECIMAL(10,2)	NOT NULL

sale_items

Column	Type	Constraints
sale_item_id	INT	PK, AUTO_INCREMENT
sale_id	INT	FK → sales(sale_id), NOT NULL
product_id	INT	FK → products(product_id), NOT NULL
quantity	DOUBLE	NOT NULL
subtotal	DECIMAL(10,2)	NOT NULL

FOREIGN KEY RULES

- sales.customer_id
  → ON DELETE SET NULL
  → ON UPDATE CASCADE
- sale_items.sale_id
  → ON DELETE CASCADE
  → ON UPDATE CASCADE
- sale_items.product_id
  → ON DELETE RESTRICT
  → ON UPDATE CASCADE

6.TEST CASES

The system was tested using multiple test cases to ensure correctness and reliability. Both valid and invalid inputs were tested.

Test Case	Input	Expected Result	Actual Result
Valid Login	correct credentials	login success	Pass
Add new user	Invalid password	Error message	Pass
Add new user	Different password and confirm password	Error message	Pass

Search user with user id	Enter non integer, enter unregistered user id	Error message	Pass
Add sale	Input invalid customer id, product id or quantity	Error message	Pass
Add product	Input invalid name, stock, price	Error message	Pass
Add product	Valid name, stock and price	Add product success	Pass
Add customer	Valid information	Add customer success	Pass
Update customer's contact number	Invalid phone number (digits less than 10)	Error message	Pass
Update customer's contact number	Unregistered customer id	Error message pop up	Pass

7.CHALLEGES AND LIMITATIONS

Challenges

- The number of classes were not kept in check due to being an unexperienced programmer, which led to an unmanageable number of classes (> 30) and packages (>5).
- Because of the above reason drawing the class diagram was abandoned.
- Difficulty in implementing database transactions correctly (commit and rollback handling).
- Managing relationships between sales, customers, and sale items without data inconsistency.
- Designing the GUI using absolute positioning (null layout), leading to alignment and scaling issues.
- Handling dynamic table updates (refreshing JTable after data changes).
- Ensuring proper validation for user input (e.g., contact numbers, empty fields, entering String instead of numbers).
- Managing multiple DAO interactions within a single operation (e.g., adding sale + sale items).
- Figuring out a method to delete records of tables with foreign key references from other tables (e.g., Product table records cannot be deleted because of the references being done by other tables).

Limitations

- The system does not include advanced security features such as password encryption.
- No connection pooling is implemented, which may affect performance in larger systems.
- GUI design is basic and not responsive to different screen sizes.
- Java swing was used to make the GUI for simplicity but it lacks the modern elements that JavaFx has.
- GUI may look cluttered due to the unfamiliarity the creator has with GUI creation.
- Limited validation rules for some inputs sometimes may give no response (no error message) due to the lack of robust error checking (only basic checks implemented).
- No support for concurrent users or multi-user environment handling.
- Error handling is basic and may not cover all edge cases.

8.CONCLUSION

The Sales Management System successfully achieves its primary objectives by providing a functional platform to manage users, customers, products and sales transactions.

plethora of validation methods are implemented to ensure the reliability of the system. Even though the limitations and challenges faced in regards of security and interface design leave much to be desired the system performs the intended tasks consistently.